

University of Moratuwa
Faculty of Engineering

EN3150

Pattern Recognition

Assignment 03: Resource-Constrained CNN for Edge Image Classification

Index No.	Name
230478C	K.C.S. Perera
230458P	H.H. Palihena
230526A	R.A.C.D. Ranawaka
230399J	M.M.M. Marshad

Contents

1 Data Preparation **2**

2 Custom Architecture Design **2**
2.1 Model A: Standard CNN Architecture 2
2.2 Model B (Lightweight CNN) 3

3 Optimizer Selection and Hyperparameter Tuning **3**

4 Custom Model Training & Evaluation **5**

5 SOTA Fine-Tuning & Evaluation **5**

6 Final Comparison & Trade-off Discussion **5**

7 GitHub Profile **5**

1 Data Preparation

For this assignment, the **Jute Pest dataset** was selected. This dataset represents a highly practical agricultural edge-computing scenario (e.g., deploying a remote IoT sensor to detect crop diseases), containing 7,240 RGB images categorized into 17 distinct classes of insect pests. To emulate a low-resource sensor environment and adhere to the strict assignment requirements, all images were downscaled to a maximum resolution of 64×64 pixels using PyTorch's `torchvision.transforms`. Additionally, the images were converted to tensors and normalized using standard ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]) values to ensure stable and efficient model training.

The originally downloaded dataset contained pre-existing train, validation, and test folders that did not conform to the required data split ratio. To resolve this and satisfy the constraints, the existing folders were loaded using `ImageFolder` and combined into a single unified dataset using PyTorch's `ConcatDataset`. The unified dataset was then randomly partitioned using a fixed manual seed (for reproducibility) into the exact required proportions:

- **Training Set (70%):** 5,068 images used for model optimization.
- **Validation Set (15%):** 1,086 images used for hyperparameter tuning and preventing overfitting.
- **Testing Set (15%):** 1,086 images reserved purely for final evaluation.

Finally, PyTorch `DataLoader` instances were created for each split with a batch size of 32 to efficiently batch and shuffle the data before feeding it into the convolutional neural networks.

2 Custom Architecture Design

2.1 Model A: Standard CNN Architecture

A standard Convolutional Neural Network (CNN) architecture was designed to establish a baseline for spatial feature extraction and image classification under constrained conditions. The architecture employs a sequential topology, systematically interleaving 2D convolutional layers with max-pooling operations to form a deep feature hierarchy.

Architectural Design Choices

The network relies on a progression of three convolutional blocks, strategically designed to balance expressive capacity with computational efficiency.

- **Kernel Size and Filter Progression:** All convolutional layers utilize 3×3 kernels. This is the smallest spatial window capable of capturing essential structural concepts (such as edges and corners) symmetrically, which minimizes the parameter count while enabling deep, non-linear feature representations. To compensate for the loss of spatial resolution deeper in the network, the channel depth is progressively doubled ($16 \rightarrow 32 \rightarrow 64$). This standard design pattern allows the network to encode increasingly complex, high-level semantic features in the channel dimension.
- **Spatial Downsampling and Receptive Field:** Each convolution is followed by a 2×2 max-pooling operation with a stride of 2. This aggressively expands the network's receptive field without requiring large, parameter-heavy kernels. Max-pooling also introduces local translation invariance and systematically shrinks the feature map (from 64×64 down to 8×8), drastically reducing the computational burden on the dense classification head.

- **Classification Head:** The spatially compressed $64 \times 8 \times 8$ feature maps are flattened into a 4,096-dimensional vector and passed through a densely connected bottleneck layer (128 units) to aggregate global context before outputting the final 17 class probabilities.

Network Topology and Parameter Count

The precise layer shapes and trainable parameters are detailed below. The parameters for convolutional layers are calculated via $((C_{in} \times K_h \times K_w) + 1) \times C_{out}$, and for fully connected (FC) layers via $(N_{in} + 1) \times N_{out}$.

- **Block 1:** `Conv2D` (3×3, 16 filters, padding=1) $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2D` (2×2). *Parameters:* $((3 \times 3 \times 3) + 1) \times 16 = 448$
- **Block 2:** `Conv2D` (3×3, 32 filters, padding=1) $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2D` (2×2). *Parameters:* $((16 \times 3 \times 3) + 1) \times 32 = 4,640$
- **Block 3:** `Conv2D` (3×3, 64 filters, padding=1) $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2D` (2×2). *Parameters:* $((32 \times 3 \times 3) + 1) \times 64 = 18,496$
- **FC1 (Bottleneck):** `Flatten` (4096 units) $\rightarrow$ `Dense` (128 units, `ReLU`). *Parameters:* $(4096 + 1) \times 128 = 524,416$
- **FC2 (Output):** `Dense` (17 units). *Parameters:* $(128 + 1) \times 17 = 2,193$

Total Trainable Parameters: 550,193.

Hardware-Aware Activation Functions

To optimize execution for embedded edge inference, the Rectified Linear Unit (ReLU) activation function, defined as $f(x) = \max(0, x)$, was selected for all hidden layers. Unlike sigmoidal functions (`Sigmoid`, `Tanh`), which require evaluating complex exponential components (e^x), ReLU relies purely on a simple hardware comparator to threshold negative values at zero. This design minimizes Arithmetic Logic Unit (ALU) clock cycles and avoids reliance on large lookup tables or floating-point coprocessors, making it highly efficient for low-resource microcontrollers.

2.2 Model B (Lightweight CNN)

3 Optimizer Selection and Hyperparameter Tuning

To establish an optimal training protocol for the resource-constrained classification architectures, an empirical ablation study was conducted using the baseline Model A. Three distinct optimization schemes were evaluated over 20 epochs using an identical cross-entropy loss function: **standard Stochastic Gradient Descent** (SGD, $\eta = 0.01$), **SGD augmented with Classical Momentum** ($\eta = 0.01$, $\beta = 0.9$), and **Adaptive Moment Estimation** (Adam, $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$).

Impact of the Momentum Parameter on Convergence Dynamics

Standard SGD updates parameters proportionally only to the current stochastic gradient estimate:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{1}$$

As evidenced by the training log, standard SGD exhibited **sluggish convergence**, dropping slowly from an initial validation loss of 2.7573 to 1.9144 by epoch 20. Because mini-batch gradient updates are inherently noisy, vanilla SGD **oscillates heavily** across steep ravines in the loss landscape, impeding rapid descent along directions of shallow curvature.

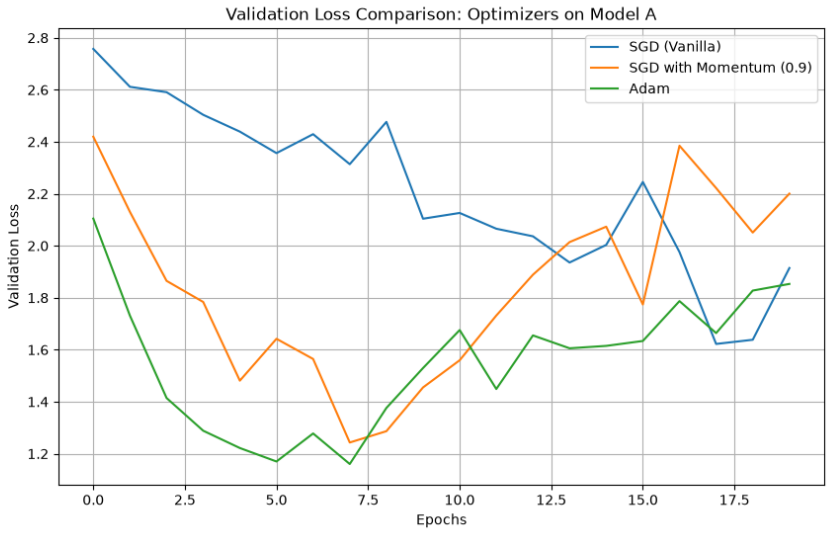


Figure 1: Validation loss trajectories across 20 training epochs for vanilla SGD, SGD with Momentum, and Adam on Model A.

Incorporating a momentum parameter ($\beta = 0.9$) introduces an **exponentially decaying moving average** of past gradient vectors:

$$v_{t+1} = \beta v_t + \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{2}$$

$$\theta_{t+1} = \theta_t - v_{t+1} \tag{3}$$

The momentum term **acts as physical inertia**: it amplifies velocity along dimensions where directional gradients consistently align while dampening high-frequency orthogonal oscillations. This mechanism **accelerated early convergence drastically** compared to standard SGD, reducing the validation loss from 2.4196 down to a minimum of 1.2432 by epoch 8 (reaching a training loss of 0.8330).

Comparative Evaluation and Optimizer Justification

While SGD with momentum demonstrated strong descent velocity, the Adam optimizer consistently exhibited the **highest sample efficiency and numerical stability**. Adam combines the directional acceleration of first-moment momentum with **per-parameter adaptive learning rates** derived from running estimates of the uncentered second raw moment:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{4}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{5}$$

As illustrated in Figure 1, Adam achieved **immediate, steep convergence** within the first five epochs, dropping the validation loss from 2.1046 to 1.2219 by epoch 5, and achieving an **absolute minimum validation loss of 1.1604** at epoch 8. Across the entire training duration, Adam sustained the lowest overall validation loss trajectory.

Notably, both SGD with Momentum and Adam began exhibiting **signs of overfitting** beyond epoch 8–10, where training losses approached near-zero values (0.0516 for Adam and 0.1471 for SGD with Momentum at epoch 20) while validation losses drifted upward due to capacity saturation on downsampled images. However, Adam's per-coordinate scaling **prevented the sharp destabilizing spikes** observed in SGD with Momentum (e.g., the validation loss surge

to 2.3849 at epoch 17). Consequently, **Adam with an initial learning rate of** $\eta = 0.001$ was chosen as the primary optimization strategy for the complete training and benchmarking of both Model A and Model B.

4 Custom Model Training & Evaluation

5 SOTA Fine-Tuning & Evaluation

6 Final Comparison & Trade-off Discussion

7 GitHub Profile